

```

def get_moves(state):
    """Generates all valid next states by moving the empty tile (0)."""
    moves = []
    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)

    # Define possible directional shifts for the blank tile index
    # (row_change, col_change, move_name)
    directions = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]

    for dr, dc, move_name in directions:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col
            # Convert to list to swap items, then convert back to tuple
            new_state = list(state)
            new_state[zero_idx], new_state[new_idx] = new_state[new_idx], new_state[zero_idx]
            moves.append((tuple(new_state), move_name))

    return moves

def depth_limited_search(state, goal, depth, path, visited):
    """Performs a Depth-Limited Search up to the specified depth."""
    if state == goal:
        return path
    if depth <= 0:
        return None

    visited.add(state)

    for next_state, move in get_moves(state):
        if next_state not in visited:
            # Explore deeper with the remaining depth allocation
            result = depth_limited_search(next_state, goal, depth - 1, path + [move], visited)
            if result is not None:
                return result

    # Backtrack to allow state evaluation along alternative paths
    visited.remove(state)
    return None

def iterative_deepening_search(start, goal):
    """Iteratively increases the depth limit until a solution is found."""
    depth = 0
    while True:
        visited = set()
        result = depth_limited_search(start, goal, depth, [], visited)
        if result is not None:
            return result
        depth += 1

def print_board(state):
    """Helper function to print the board layout cleanly."""
    for i in range(0, 9, 3):
        print(f"{state[i]} {state[i+1]} {state[i+2]}")
    print("-" * 5)

# --- Example Execution ---
if __name__ == "__main__":
    # 0 represents the empty/blank space
    # Standard Goal State
    goal_state = (1, 2, 3,
                  8, 0, 4,
                  7, 6, 5)

    # An example start state (solvable in 4 moves)
    start_state = (2, 8, 3,
                  1, 6, 4,
                  7, 0, 5)

    print("Initial State:")
    print_board(start_state)

    print("Searching for solution using IDS...")
    solution_path = iterative_deepening_search(start_state, goal_state)

    if solution_path:
        print(f"Goal Reached successfully in {len(solution_path)} moves!")
        print("Sequence of moves:", " -> ".join(solution_path))

```

```
else:  
    print("No solution path could be found.")
```

Initial State:

2 8 3

1 6 4

7 0 5

Searching for solution using IDS...

Goal Reached successfully in 5 moves!

Sequence of moves: Up -> Up -> Left -> Down -> Right